

Intelligent Security API

Developer Guide

dojojin@gmail.com
VISION AND SECURITY SYSTEM CO.,LTD.

About this Document

- This Document includes instructions for using and managing the Product. Pictures, charts, images and all other information hereinafter are for description and explanation only. Unless otherwise agreed, our company makes no warranties, express or implied.
- Please use this Document with the guidance and assistance of professionals trained in supporting the Product.

Trademarks Acknowledgment

All trademarks and logos mentioned are the properties of their respective owners.

LEGAL DISCLAIMER

- TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THIS DOCUMENT AND THE PRODUCT DESCRIBED, WITH ITS HARDWARE, SOFTWARE AND FIRMWARE, ARE PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". OUR COMPANY MAKES NO WARRANTIES, EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION, MERCHANTABILITY, SATISFACTORY QUALITY, OR FITNESS FOR A PARTICULAR PURPOSE. THE USE OF THE PRODUCT BY YOU IS AT YOUR OWN RISK. IN NO EVENT WILL OUR COMPANY BE LIABLE TO YOU FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION, OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), PRODUCT LIABILITY, OR OTHERWISE, IN CONNECTION WITH THE USE OF THE PRODUCT, EVEN IF OUR COMPANY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.
- YOU ACKNOWLEDGE THAT THE NATURE OF THE INTERNET PROVIDES FOR INHERENT SECURITY RISKS, AND OUR COMPANY SHALL NOT TAKE ANY RESPONSIBILITIES FOR ABNORMAL OPERATION, PRIVACY LEAKAGE OR OTHER DAMAGES RESULTING FROM CYBER-ATTACK, HACKER ATTACK, VIRUS INFECTION, OR OTHER INTERNET SECURITY RISKS; HOWEVER, OUR COMPANY WILL PROVIDE TIMELY TECHNICAL SUPPORT IF REQUIRED.
- YOU AGREE TO USE THIS PRODUCT IN COMPLIANCE WITH ALL APPLICABLE LAWS, AND YOU ARE SOLELY RESPONSIBLE FOR ENSURING THAT YOUR USE CONFORMS TO THE APPLICABLE LAW. ESPECIALLY, YOU ARE RESPONSIBLE, FOR USING THIS PRODUCT IN A MANNER THAT DOES NOT INFRINGE ON THE RIGHTS OF THIRD PARTIES, INCLUDING WITHOUT LIMITATION, RIGHTS OF PUBLICITY, INTELLECTUAL PROPERTY RIGHTS, OR DATA PROTECTION AND OTHER PRIVACY RIGHTS. YOU SHALL NOT USE THIS PRODUCT FOR ANY PROHIBITED END-USES, INCLUDING THE DEVELOPMENT OR PRODUCTION OF WEAPONS OF MASS DESTRUCTION, THE DEVELOPMENT OR PRODUCTION OF CHEMICAL OR BIOLOGICAL WEAPONS, ANY ACTIVITIES IN THE CONTEXT RELATED TO ANY NUCLEAR EXPLOSIVE OR UNSAFE NUCLEAR FUEL-CYCLE, OR IN SUPPORT OF HUMAN RIGHTS ABUSES.
- IN THE EVENT OF ANY CONFLICTS BETWEEN THIS DOCUMENT AND THE APPLICABLE LAW, THE LATTER PREVAILS.

VISION AND SECURITY SYSTEM CO., LTD.
doc@jovision.com

1 Reading Guide

Chapter	Description
Overview	Includes the ISAPI overview, applicable products, terms and definitions, abbreviations, and update history.
ISAPI Framework	Read the chapter to take a quick look at the ISAPI framework and basic functions.
Quick Start Guide	Read the chapter to quickly understand the programming process of basic functions such as authentication, message parsing, real-time live view, playback, and event uploading.
API Reference	Start programming according to API definitions.
How-To Video Guidance	How-to videos demonstrate detailed steps of different integration tasks.

2 Overview

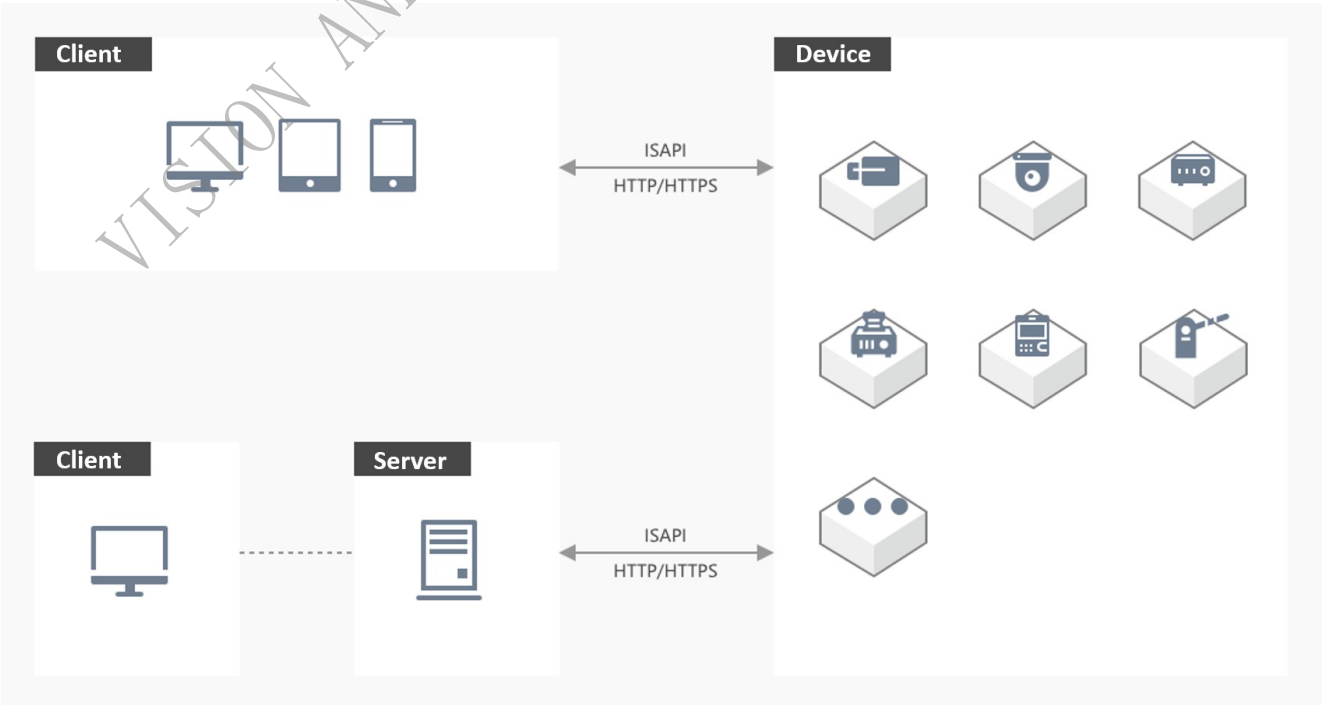
2.1 Introduction

Intelligent Security API (hereinafter referred to as ISAPI) is an application layer protocol based on HTTP (Hypertext Transfer Protocol) and adopts the REST (Representational State Transfer) architecture for communication between security devices (cameras, DVRs, NVRs, etc.) and the platform or client software.

Since established in 2013, ISAPI has included more than 11,000 APIs for different functions, including device management, vehicle recognition, parking lot management, intelligent facial application, access control management, interrogation management, and recording management. It is applicable to industries such as traffic, fire protection, education, and security inspection.

2.1.1 Application Scenario

When you integrate devices via ISAPI, the device acts as the server to listen on the fixed port and the user's application acts as the client to actively log in to the device for communication. To achieve the above goals, the device should be configured with a fixed IP address and the requests from the client can reach the server.



VISION AND SECURITY SYSTEM CO.,LTD. dojojing@gmail.com

2.1.2 Layers in the Network Model

ISAPI is an application layer protocol based on HTTP, thereby it inherits all specifications and properties from HTTP.

Protocols frequently used along with ISAPI include SADP (Search Active Device Protocol) based on multicast for discovering and activating devices, RTSP (Real-Time Streaming Protocol) based on TCP/UDP for live view and video playback of devices, etc.

<table>
<tr><td rowspan="2">Application Layer</td><td>Device Discovery</td><td>Signaling Interaction</td><td>Media Stream</td></tr>
<tr><td>SADP</td><td>ISAPI (HTTP)</td><td>RTSP/RTP</td></tr>
<tr><td>Transport Layer</td><td>MCAST</td><td>TCP</td><td>TCP/UDP</td></tr>
<tr><td>Network Layer</td><td colspan="3">IP</td></tr>
<tr><td>Network Interface</td><td colspan="3">Hardware Drive Interface</td></tr>
</table>

2.2 Product Scope

- DeepinMind AI Products
  - DeepinMind Edge AcuSeek

    iDS-6716NXI-P1, iDS-7608NXI-P2, iDS-7616NXI-P2, iDS-7716NXI-P4, iDS-7716NXI-P4/16P, iDS-7732NXI-P4, iDS-7732NXI-P4/16P, iDS-9616NXI-P8, iDS-9616NXI-P8R, iDS-9632NXI-P16, iDS-9632NXI-P16R, iDS-9632NXI-P8, iDS-9632NXI-P8R, iDS-9664NXI-P16, iDS-9664NXI-P16R, iDS-9664NXI-P8, iDS-9664NXI-P8R

2.3 Terms And Definitions

There are no terms or definitions in this document.

2.4 Symbols And Acronyms

FDLib: Face Picture Library

FDID: Face Picture Library ID

PID: Face Picture ID in the Face Picture Library

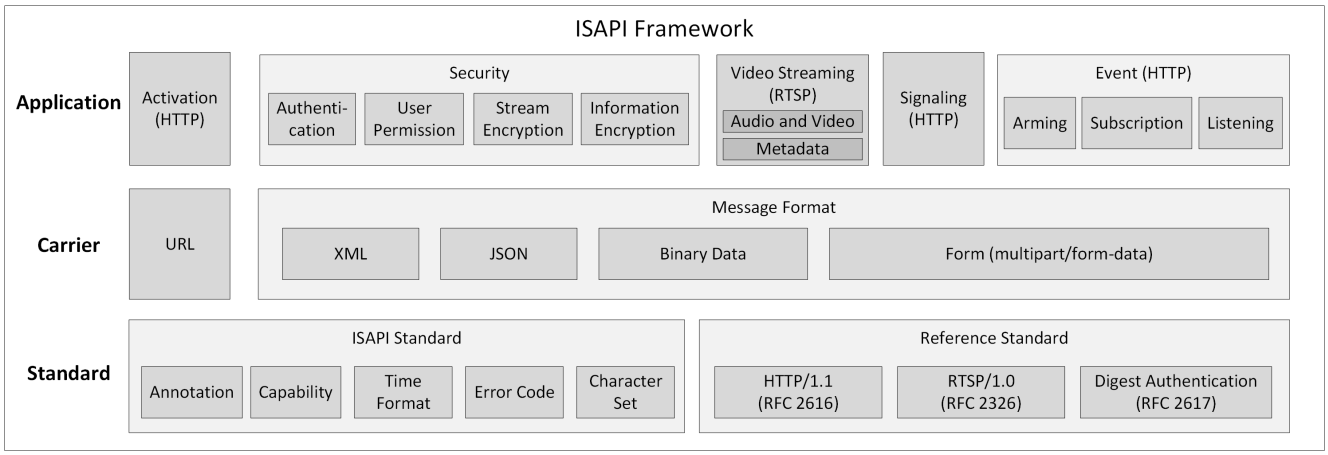
2.5 Update History

No update record

3 ISAPI Framework

3.1 Overview

VISION AND SECURITY SYSTEM CO.,LTD. dojo jing@gmail.com



Notes:

In general, ISAPI refers to the communication protocol based on the HTTP standard. As ISAPI is usually used along with RTSP (Real-Time Streaming Protocol), the RTSP standard is brought into the ISAPI system.

The metadata scheme for transmitting additional information of the stream is extended based on the RTSP standard to transmit the video stream and the structured intelligent information of the stream simultaneously. It is compatible with the RTSP standard.

3.2 Activation

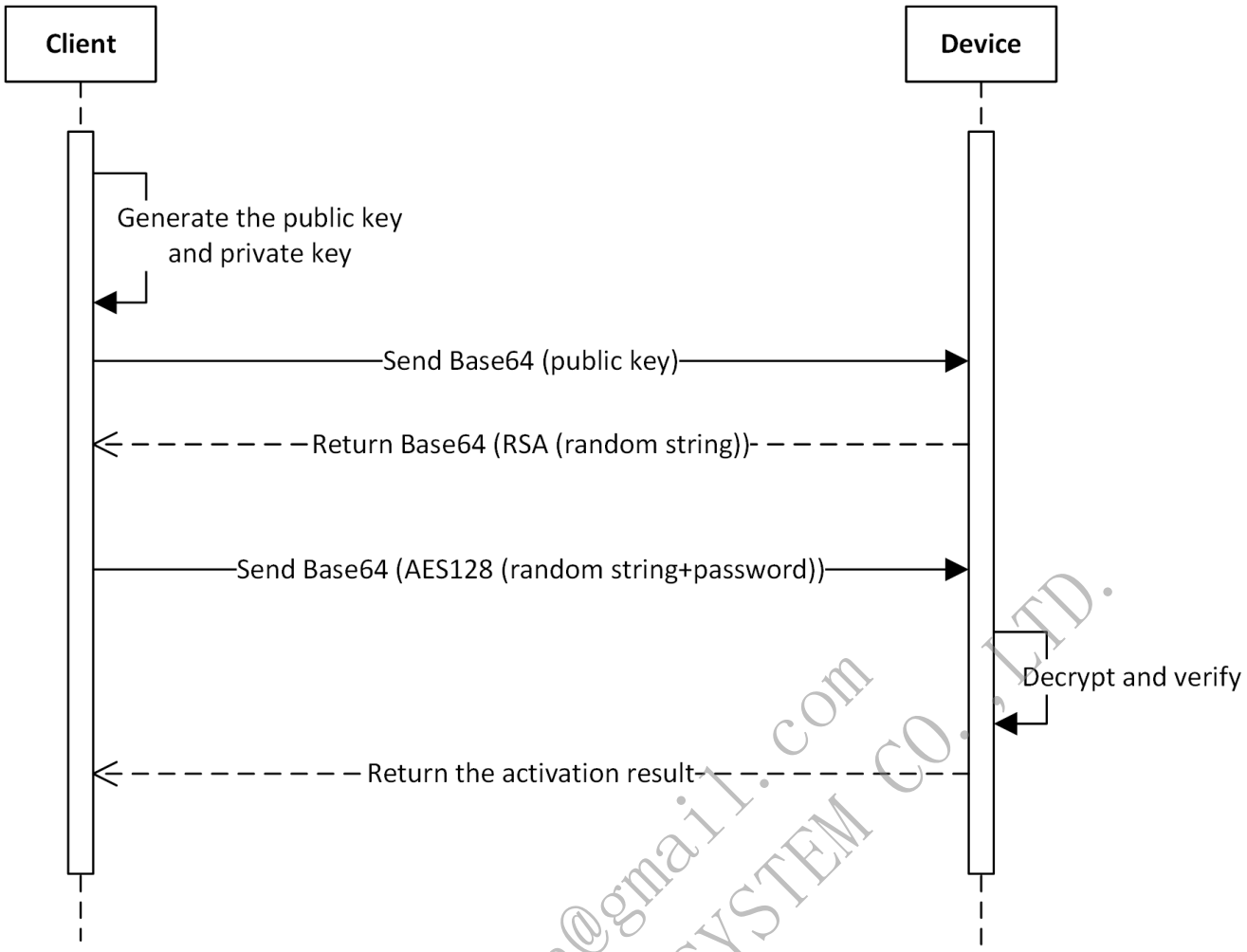
The purpose of activation is to ensure that the user can set the password for the device and the password meets the security requirement. After the device is activated, you can use the related functions.

ISAPI is a communication protocol running on the application layer. When activating the device via ISAPI, you should know the device's IP address and make sure that the device is connected to the client.

The web application built in the device supports activating the device via ISAPI. When you enter the device's IP address in the address bar of the web browser on the PC, you can activate the device according to the activation guide.

If you want to activate the device on your own application, you need to integrate the activation function via ISAPI. The API calling flow and related APIs are shown below.

dojoin@gmail.com
VISION AND SECURITY SYSTEM CO., LTD.



VISION AND SECURITY SYSTEM CO., LTD. dojojin@gmail.com

Firstly, two operations are defined:

- bytesToHexstring: it is used to convert a byte array (the length is N) to a hexadecimal string (the length is 2N). For example, `127,10,23` -> `7f0a17`
- hexStringToBytes: it is used to convert a hexadecimal string (the length is 2N) to a byte array (the length is N). For example, `7f0a17` -> `127,10,23`

1. The client generates a public and private key pair (1024 bits), and gets the 128-byte modulus in the public key (hereinafter referred to as public key modulus). If the length is longer than 128, the leading 0 needs to be removed.
2. The client converts the public key modulus to a 256-byte public key string via bytesToHexstring and sends the public key string to the device in XML message (related URI: `POST /ISAPI/Security/challenge`) after being encoded by Base64.
3. The device parses the request to obtain a 256-byte public key string decoded by Base64 and converts it to a 128-byte public key modulus via hexStringToBytes. The complete public key is the combination of obtained public key modulus and public exponent (the default value is `'010001'`).
4. The device generates a 32-byte hexadecimal random string, calls the RSA API to encrypt the random string with the private key, converts the encrypted data to a string via bytesToHexstring, encodes the string by Base64, and then sends it to the client.
5. The client decodes the string from the device by Base64, converts it via hexStringToBytes to get the encrypted data, decrypts the encrypted data with the private key via RSA to obtain a 32-byte hexadecimal random string, converts the obtained string via hexStringToBytes to get a 16-byte AES key. Then the client uses the AES key to encrypt the `"string consisting of the first 16 characters of the random string and the real password"` by AES128 ECB mode (with zero-padding method) to get a ciphertext, and converts the ciphertext via bytesToHexstring, encodes it by Base64, and sends it to the device in XML message (related URI: `PUT /ISAPI/System/activate`). Note: If the first 16 characters of the random string are `aaaabbbbccccdddd` and the real password is `Abc12345`, the data before encryption is `aaaabbbbccccddddAbc12345`. This can ensure that the client uses the random string as the key for encryption.
6. The device decodes the string by Base64, converts it via hexStringToBytes to get the ciphertext, uses the AES key to decrypt the ciphertext by AES128 ECB mode, and gets the real password via removing the first 16 characters.

7. The device verifies the password and returns the activation result.

Notes:

- You can get the device's activation status by calling the URI `GET /SDK/activateStatus` which requires no authentication.
- Devices also support to be activated via SADP (Search Active Device Protocol) which is based on the communication protocol of the data link layer. With SADP, you do not have to know the IP address of the device but need to ensure that the device and the application running SADP are connected to the same router. SADP also supports discovering devices in the LAN, changing the password of the devices, and so on. The HCSadpSDK is provided for SADP integration, including the developer guide, plug-in, and sample demo which can be used as a simple SADP tool.

3.3 Security Mechanism

3.3.1 Authentication

When the client applications send requests to devices, they need to use digest authentication (see details in RFC 7616) for identity authentication.

Currently, all mainstream request class libraries of HTTP have encapsulated digest authentication. See details in Authentication of Quick Start Guide.

3.3.2 User Permission

There are three kinds of users with different permissions for access control and management.

Administrator: Has the permission to access all supported resources and should keep activated all the time. It is also known as "admin".

Operator: Has the permission to access general resources and a part of advanced resources.

Normal User: Only has the permission to access general resources.

3.3.3 Information Encryption

During ISAPI integration, the HTTPS service of devices is enabled by default. When the client applications communicate with devices via HTTPS, the information can be transmitted securely.

3.4 Video Streaming

3.4.1 Audio and Video Stream

ISAPI supports getting and setting stream media parameters of the device, such as video resolution, encoding format, and stream.

Cameras support standard RTSP (Real-Time Streaming Protocol, see details in RFC 7826). Client applications can get the stream from devices via RTSP.

For details about real-time streaming and video playback, refer to **Real-Time Live View** and **Playback** in Quick Start Guide.

3.4.2 Metadata

The metadata is the structured intelligent information generated by intelligent devices. When the client applications get the audio and/or video stream from devices via RTSP, the metadata will be returned by the device at the same time. For example, to display the face target frame, face information, vehicle target frame, license plate number, vehicle information, and other information on the video stream, the client applications can overlay the above information on the video image.

Before using the metadata, you need to enable the metadata function of the device and then get the stream from the device via RTSP. Some devices support subscribing to the metadata by type. For details about the process of integrating the metadata function, refer to **Metadata Management**.

VISION AND SECURITY SYSTEM CO., LTD. dejojin@gmail.com

4 Quick Start Guide

4.1 Authentication

When the client applications send requests to the devices, they need to use digest authentication (see details in RFC 7616) for identity authentication.

Client applications only need to call APIs of the class library to implement the digest authentication. The sample code is shown below.

4.1.1 C/C++ (libcurl)

```
// #include <curl/curl.h>
// Callback Function
static size_t OnWriteData(void* buffer, size_t size, size_t nmemb, void* lpVoid)
{
    std::string* str = dynamic_cast<std::string*>((std::string *)lpVoid);
    if( NULL == str || NULL == buffer )
    {
        return -1;
    }

    char* pData = (char*)buffer;
    str->append(pData, size * nmemb);
    return nmemb;
}

std::string strUrl = "http://192.168.18.84:80/ISAPI/System/deviceInfo";
std::string strResponseData;
CURL *pCurlHandle = curl_easy_init();
curl_easy_setopt(pCurlHandle, CURLOPT_CUSTOMREQUEST, "GET");
curl_easy_setopt(pCurlHandle, CURLOPT_URL, strUrl.c_str());
// Set the user name and password
curl_easy_setopt(pCurlHandle, CURLOPT_USERPWD, "admin:admin12345");
// Set the authentication method to the digest authentication
curl_easy_setopt(pCurlHandle, CURLOPT_HTTPAUTH, CURLAUTH_DIGEST);
// Set the callback function
curl_easy_setopt(pCurlHandle, CURLOPT_WRITEFUNCTION, OnWriteData);
// Set the parameters of the callback function to get the returned information
curl_easy_setopt(pCurlHandle, CURLOPT_WRITEDATA, &strResponseData);
// Timeout settings for receiving the data. If receiving data is not completed within 5 seconds, the application will exit directly
curl_easy_setopt(pCurlHandle, CURLOPT_TIMEOUT, 5);
// Set the redirection times to avoid too many redirections
curl_easy_setopt(pCurlHandle, CURLOPT_MAXREDIRS, 1);
// Connection timeout duration. If the duration is too short, the client application will be disconnected before the data request sent by the application
reaches the device
curl_easy_setopt(pCurlHandle, CURLOPT_CONNECTTIMEOUT, 5);
CURLcode nRet = curl_easy_perform(pCurlHandle);
if (0 == nRet)
{
    // Output the received message
    std::cout << strResponseData << std::endl;
}
curl_easy_cleanup(pCurlHandle);
```

4.1.2 C# (WebClient)

```
// using System.Net;
// using System.Net.Security;
try
{
    string strUrl = "http://192.168.18.84:80/ISAPI/System/deviceInfo";
    WebClient client = new WebClient();
    // Set the user name and password
    client.Credentials = new NetworkCredential("admin", "admin12345");
    byte[] responseData = client.DownloadData(strUrl);
    string strResponseData = Encoding.UTF8.GetString(responseData);
    // Output received information
    Console.WriteLine(strResponseData);
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

4.1.3 Java (HttpClient)

VISION AND SECURITY SYSTEM CO.,LTD.
dongjin@gmail.com

```java
// import org.apache.commons.httpclient.HttpClient;
String url = "http://192.168.18.84:80/ISAPI/System/deviceInfo";
HttpClient client = new HttpClient();
// Set the user name and password
UsernamePasswordCredentials creds = new UsernamePasswordCredentials("admin", "admin12345");
client.getState().setCredentials(AuthScope.ANY, creds);
GetMethod method = new GetMethod(url);
method.setDoAuthentication(true);
int statusCode = client.executeMethod(method);
byte[] responseData = method.getResponseBodyAsString().getBytes(method.getResponseCharSet());
String strResponseData = new String(responseData, "utf-8");
method.releaseConnection();
// Output received information
System.out.println(strResponseData);
```

4.1.4 Python (requests)

```python
# - *- coding: utf-8 -*-

import requests
request_url = 'http://192.168.18.84:80/ISAPI/System/deviceInfo'
# Set the authentication information
auth = requests.auth.HTTPDigestAuth('admin', 'admin12345')
# Send the request and receive response
response = requests.get(request_url, auth=auth)
# Output response content
print(response.text)
```

4.2 Message Parsing

4.2.1 Message Format

During the process of communication and interaction via ISAPI, the request and response messages are often text data in XML or JSON format. Besides that, the data of firmware packages and configuration files is in binary format. A request can also be in form format with multiple formats of data (multipart/form-data).

4.2.1.1 XML

Generally, the `Content-Type` in the headers of the HTTP request is `application/xml; charset="UTF-8"`.

Request and response messages in XML format are all encoded with UTF-8 standards in ISAPI.

The namespace `http://www.isapi.org/ver20/XMLSchema` and ISAPI version number `2.0` of XML messages are configured by default, see the example below.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<NodeList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <Node>
    <id>1</id>
    <enabled>true</enabled>
    <nodeName>nodeName</nodeName>
    <level>level1</level>
  </Node>
</NodeList>
```

4.2.1.2 JSON

The `Content-Type` in the headers of the HTTP request is often `application/json`.

To distinguish between APIs with XML messages and those with JSON messages, ISAPI adds the query parameter format=json to all request URLs with JSON messages, e.g., `http://192.168.1.1:80/ISAPI/System/Sensor/thermometrySensor?format=json`. Messages of request URLs without the query parameter format=json are usually in XML format. However, there may be some exceptions, and the message format is subject to the API definition.

Request and response messages in JSON format are all encoded by UTF-8 in ISAPI.

4.2.1.3 Binary Data

VISION AND SECURITY SYSTEM CO.,LTD. dojojin@gmail.com

For the firmware and configuration files, the `Content-Type` in the header of an HTTP request is often `application/octet-stream`.

4.2.1.4 Form (multipart/form-data)

When multiple pieces of data are submitted at the same time in an ISAPI request (e.g., the person information and face picture need to be submitted at the same time when a face record is added to the face picture library), the `Content-Type` in the header of the corresponding HTTP request is usually `multipart/form-data, boundary=AaB03x`, where the boundary is a variable used to separate the entire HTTP body into multiple units and each unit is a piece of data with its own headers and body. In `Content-Disposition` of form unit headers, the `name` property refers to the form unit name, which is required for all form units; the `filename` property refers to the file name of form unit body, which is required only when the form unit body is a file. In headers of form units, `Content-Length` refers to the body length, which starts after CRLF(`\r\n`) and ends before two hyphens (`--`) of next form. There should be a CRLF used as the delimiter of two form units before two hyphens (`--`), and the `Content-Length` of previous form unit does not include the CRLF length. For the detailed format description, refer to RFC 1867 (Form-Based File Upload in HTML). Pay attention to two hyphens (`--`) before and after the boundary.

Notes

- In RFC specifications, it is strongly recommended to contain the field `Content-Length` in the entity header, and there is no requirement that the field `Content-Length` should be contained in the header of each form element. The absence of field `Content-Length` in the header should be considered when the client and device programs parse the form data.
- To avoid the conflict between message content and boundary value, it is recommended to use a longer and more complex string as the boundary value.

The example of ISAPI form data submitted by a client to a device is as follows.

```
POST /ISAPI/Intelligent/FDLib/pictureUpload
Content-Type: multipart/form-data; boundary=e5c2f8c5461142aea117791dade6414d
Content-Length: 56789

--e5c2f8c5461142aea117791dade6414d
Content-Disposition: form-data; name="PictureUploadData";
Content-Type: application/xml
Content-Length: 1234

<PictureUploadData/>
--e5c2f8c5461142aea117791dade6414d
Content-Disposition: form-data; name="face_picture"; filename="face_picture.jpg";
Content-Type: image/jpeg
Content-Length: 34567

Picture Data
--e5c2f8c5461142aea117791dade6414d--
```

The example of ISAPI form data responded from a device to a client is as follows.

In ISAPI messages, when there are multiple form units, three nodes (`pid`, `contentid`, and `filename`) are used for linking form units. The corresponding relations are as follows:

Node Name	Form Field	Description
pid	name	pid in XML/JSON messages corresponds to the name property of Content-Disposition in form headers.
contentid	Content-ID	contentid in XML/JSON messages corresponds to Content-ID in form headers.
filename	filename	filename in XML/JSON messages corresponds to filename property of Content-Disposition in form headers.

VISION AND SECURITY SYSTEM CO., LTD. dojojin@gmail.com

```
HTTP/1.1 200 OK
Content-Type: multipart/form-data; boundary=136a73438ecc4618834b999409d05bb9
Content-Length: 56789

--136a73438ecc4618834b999409d05bb9
Content-Disposition: form-data; name="mixedTargetDetection";
Content-Type: application/json
Content-Length: 811

{
    "ipAddress": "172.6.64.7",
    "macAddress": "01:17:24:45:D9:F4",
    "channelID": 1,
    "dateTime": "2009-11-14T15:27+08:00",
    "eventType": "mixedTargetDetection",
    "eventDescription": "Mixed target detection",
    "deviceID": "123456789",
    "CaptureResult": [{
        "targetID": 1,
        "Human": {
            "Rect": {
                "height": 1.0,
                "width": 1.0,
                "x": 0.0,
                "y": 0.0
            },
            "contentID1": "humanImage", /*human body thumbnail*/
            "contentID2": "humanBackgroundImage", /*human body background picture*/
            "pId1": "9d48a26f7b8b4f2390c16808f93f3534", /*human body thumbnail ID */
            "pId2": "5EE7078E07BB47CF860DE8E4E9A85F28" /*ID of human body background picture*/
        }
    }]
}
--136a73438ecc4618834b999409d05bb9
Content-Disposition: form-data; name="9d48a26f7b8b4f2390c16808f93f3534"; filename="humanImage.jpg";
Content-Type: image/jpeg
Content-Length: 34567
Content-ID: humanImage

Picture Data
--136a73438ecc4618834b999409d05bb9
Content-Disposition: form-data; name="5EE7078E07BB47CF860DE8E4E9A85F28"; filename="humanBackgroundImage.jpg";
Content-Type: image/jpeg
Content-Length: 345678
Content-ID: humanBackgroundImage

Picture Data
--136a73438ecc4618834b999409d05bb9--
```

4.2.2 Annotation

The field descriptions of ISAPI request and response messages are marked as annotations in the example messages as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>

<NodeList xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, node list, attr:version{req, string, version No., range:[,]}-->
  <Node>
    <!--ro, opt, object, node information-->
    <id>
      <!--ro, req, int, node No., range:[,], step:, unit:, unitType:-->1
    </id>
    <enabled>
      <!--ro, opt, bool, whether to enable-->true
    </enabled>
    <nodeName>
      <!--ro, req, string, node name, range:[1,32]-->test
    </nodeName>
    <level>
      <!--ro, opt, enum, level, subType:string,
      [level1#Level 1,level2#Level 2,level3#Level 3]-->level1
    </level>
  </Node>
</NodeList>
```

dojojin@gmail.com VISION AND SECURITY SYSTEM CO.,LTD.

```
{
    "name":  "test",
    /*ro, req, string, name, range:[1,32]*/
    "type":  "type1",
    /*ro, req, enum, type, subType:string, [type1#type 1,type2#type 2]*/
    "enabled":  true,
    /*ro, opt, bool, enable or not, desc:xxxxxxx*/
    "NodeList": {
    /*opt, object, node list, dep:and,{$.enabled,eq,true}*/
        "scene":  1,
        /*req, enum, scene, subType:int, [1#scene 1; 2#scene 2; 3#scene 3]*/
        "ID":  1
        /*req, int, No., range:[1,8], step:, unit:, unitType:*/
    }
}
```

Key annotations are shown in the table below.

Annotation	Description	Remark
ro	Attribute: Read-Only	This field can only be obtained and cannot be edited.
wo	Attribute: Write-Only	This field can only be edited and cannot be obtained.
req	Attribute: Required	This field is required for request messages sent to the device and response messages returned from the device.
opt	Attribute: Optional	This field is optional for request messages sent to the device and response messages returned from the device.
dep	Attribute: Dependent	This field is valid and required when specific conditions are satisfied.
object	Field Type: Object	The field of type object contains multiple sub-fields.
list	Field Type: List	The subType following it refers to the data type of sub-items in the list.
subType	Field Type: String	The range following it refers to the maximum and the minimum string size of the field.
int	Field Type: Int	The range following it refers to the maximum and the minimum value of the field.
float	Field Type: Float	The range following it refers to the maximum and the minimum value of the field.
bool	Field Type: Boolean	The value can be true or false.
enum	Field Type: Enumeration	The subType following it indicates that the enumerators are of type string or int. The [] following the subType contains all enumerators.
subType	Sub-Type of Field	When the type of field is list or enum, the value of subType is the data type of each sub-object.
desc	Field Description	The detailed description of the field.

VISION AND SECURITY SYSTEM CO.,LTD. dojo.jin@gmail.com

4.2.3 Capability Set

ISAPI has designed capability sets for almost all functions, APIs, and fields. URLs for getting the capability set end with `/capabilities`. Some URLs may contain query parameters in the format: `/capabilities?format=json&type=xxx`.

There are two types of fields in the capability message of ISAPI: whether the device supports a function and the value range of a field in an API.

Whether the device supports a function: it is often in the format `isSupportXxxxxxxx`, which indicates that whether the device supports a function and a set of APIs for implementing this function.

The capability message example in JSON format is shown below:

```
{
    "isSupportMap":  true,
    /*ro, opt, bool, whether it supports the e-map function, desc:/ISAPI/SDT/Management/map/capabilities?format=json*/
    "isSupportAlgTrainResourceInfo":  true,
    /*ro, opt, bool, whether it supports only getting the resource information of the algorithm training platform,
desc:/ISAPI/SDT/algorithmTraining/ResourceInfo?format=json*/
    "isSupportAlgTrainAuthInfo":  true,
    /*ro, opt, bool, whether it supports ony getting the authorization information of the algorithm training platform,
desc:/ISAPI/SDT/algorithmTraining/SoftLock/AuthInfo?format=json*/
    "isSupportAlgTrainNodeList":  true,
    /*ro, opt, bool, whether it supports only getting the node information of the algorithm training platform, desc:/ISAPI/SDT/algorithmTraining/NodeList?
format=json*/
    "isSupportNAS":  true
    /*ro, opt, bool, whether it supports mounting and unmounting NAS, desc:/ISAPI/SDT/Management/NAS/capabilities?format=json*/
}
```

The capability message example in XML format is shown below:

```
<isSupportNetworkStatus>
    <!--ro, opt, bool, whether it supports searching the network status, desc: related API (/ISAPI/System/Network/status?format=json)-->true
</isSupportNetworkStatus>
```

The value range of the field: the maximum value, minimum value, the maximum size, the minimum size, options, and so on of each field of the API.

The example of JSON format is shown below:

```
{
    "boolType": {
        /*req, object, example of the capability of type bool*/
        "@opt": [true, false]
        /*req, array, options, subType: bool*/
    },
    "integerType": {
        /*req, object, example of the capability of type integer*/
        "@min": 0,
        /*ro, req, int, the minimum value*/
        "@max": 100
        /*ro, req, int, the maximum value*/
    },
    "stringType": {
        /*req, object, example of the capability of type string*/
        "@min": 0,
        /*ro, req, int, the minimum string size*/
        "@max": 32
        /*ro, req, int, the maximum string size*/
    },
    "enumType": {
        /*req, object, capability example of type enum*/
        "@opt": ["enum1", "enum2", "enum3"]
        /*req, array, options, subType: string*/
    }
}
```

The example of XML format is shown below:

```
<boolType opt="true,false" def="true">
    <!--ro, opt, bool, example of the capability of type bool-->true
</boolType>
<integerType min="0" max="100">
    <!--ro, opt, int, example of the capability of type int-->50
</integerType>
<stringType min="0" max="64">
    <!--ro, opt, string, example of the capability of type string-->test
</stringType>
<enumType opt="red,white,black" def="red">
    <!--ro, opt, string, example of the capability of type enum-->white
</enumType>
```

Note: For the same capability set, devices of different models and versions may return different results. The values shown in this document are only examples for reference. The capability set actually returned by the device takes precedence.

4.2.4 Time Format

ISAPI adopts ISO 8601 Standard Time Format, which is the same as W3C Standard Date and Time Formats.

VISION AND SECURITY SYSTEM CO.,LTD. dojojin@gmail.com

Format: `YYYY-MM-DDThh:mm:ss.sTZD`

```
YYYY = the year consisting of four decimal digits
MM = the month consisting of two decimal digits (01-January, 02-February, and so forth)
DD = the day consisting of two decimal digits (01 to 31)
hh = the hour consisting of two decimal digits (00 to 23, a.m. and p.m. are not allowed)
mm = the minute consisting of two decimal digits (00 to 59)
ss = the second consisting of two decimal digits (00 to 59)
s = one or more digits representing the fractional part of a second
TZD = time zone identifier (Z or +hh:mm or -hh:mm)
```

Example: 2017-08-16T20:17:06.123+08:00 refers to 20:17:06.123 on August 16, 2017 (local time which is 8 hours ahead of UTC). The plus sign (+) indicates that the local time is ahead of UTC, and the minus sign (-) means that the local time is behind UTC.

After the DST is enabled, the local time and time difference will change compared with UTC, and the values of related fields also need to be changed. Disabling the DST will bring into the opposite effect.

Example: In 1986, the DST was in effect from May 4 at 2:00 a.m. (GMT+8). During the DST period, the clocks were moved one hour ahead, which means that there was one less hour on that day. When the DST ends at 2:00 a.m. on September 14, 1986, the clocks were moved one hour back and there was an extra hour on that day. The changes of the time are as follows:

- DST Starts: 1986-05-04T02:00:00+08:00 --> 1986-05-04T03:00:00+09:00
- DST Ends: 1986-09-14T02:00:00+09:00 --> 1986-09-14T01:00:00+08:00

Notes:

- The time difference cannot be simply used to determine the time zone. Because when the DST starts, the time difference will change and it cannot represent the actual time zone.
- Both TZ (UTC time, e.g., 1986-05-03T18:00:00Z) and TD (local time and time difference, e.g., 1986-05-04T02:00:00+08:00) meet the time format standards of ISO 8601. In ISAPI, the TD format is recommended to be used in messages sent from the user applications and the devices.
- A few old-version devices will return the time in TZ format. For representing the time difference information and forward compatibility, an extra field `timeDiff` is added as shown in the example below. User applications need to support both TD format and TZ format when parsing the time in the messages returned by devices.

```
{
    "dateTime": "1986-05-03T18:00:00Z", /*device time. The value in TZ format is the UTC time and the value in TD format is the time difference between the device's local time and UTC*/
    "timeDiff": "+08:00" /*optional, time difference between the local time and UTC time. If this field does not exist, the user application will convert the dateTime into the local time for use*/
}
```

4.2.5 Character Set

To prevent characters not commonly used from resulting in exceptions in device programs and user applications, ISAPI limits the valid field values of type string to a specific range of characters. Character sets allowed to be used in the fields of type string in ISAPI are listed below.

- Single-byte character set: lowercase letters (`a-z`), uppercase letters (`A-Z`), digits (`0-9`), and special characters (see details in the table below).
- Multi-byte character set: language characters based on Unicode and encoded by UTF-8 (UTF-8 encoding is a transformation format of Unicode character set. For details, refer to RFC 2044).

VISION AND SECURITY SYSTEM CO.,LTD
dojoJing@gmail.com

No.	Name	Special Character	No.	Name	Special Character
1	Open Parenthesis	(	18	Dollar Sign	$
2	Close Parenthesis	)	19	Percent Sign	%
3	Plus Sign	+	20	Ampersand	&
4	Comma	,	21	Close Single Quotation Mark	'
5	Minus Sign	-	22	Asterisk	*
6	Period	.	23	Slash	/
7	Semicolon	;	24	Smaller Than	<
8	Equal Sign	=	25	Greater Than	>
9	At Sign	@	26	Question Mark	?
10	Open Square Bracket	[	27	Caret	^
11	Close Square Bracket	]	28	Open Single Quotation Mark	'
12	Underscore	_	29	Vertical Bar	|
13	Open Brace	{	30	Tilde	~
14	Close Brace	}	31	Double Quotation Marks	"
15	Space		32	Colon	:
16	Exclamation Mark	!	33	Backslash	|
17	Octothorpe	#			

The valid characters that can be used in some special fields are listed below.

- User name: lowercase letters (`a-z`), uppercase letters (`A-Z`), digits (`0-9`), and characters from No. 1 to No. 30 in the special character table.
- Password: User Name: lowercase letters (`a-z`), uppercase letters (`A-Z`), digits (`0-9`), and characters from No. 1 to No. 33 in the special character table.
- Names displayed on the UI (device name, person name, face picture library name, etc.): lowercase letters (`a-z`), uppercase letters (`A-Z`), digits (`0-9`), characters from No. 1 to No. 15 in the special character table, and multi-byte characters.
- Normal fields of type string support lowercase letters (`a-z`), uppercase letters (`A-Z`), digits (`0-9`), characters from No. 1 to No. 15 in the special character table, and multi-byte characters by default.

4.2.6 Error Processing

When requesting via ISAPI failed (the HTTP status code is not 200), the device will return the HTTP status code and ISAPI error code. For HTTP status codes, refer to 10 Status Code Definitions in RFC 2616. For ISAPI error codes, refer to Error Code Dictionary.

Message Example:

VISION AND SECURITY SYSTEM CO.,LTD. dc.lojin@gmail.com

```
HTTP/1.1 403 Forbidden
Content-Type: application/json; charset="UTF-8"
Date: Thu, 15 Jul 2021 20:43:30 GMT
Content-Length: 229
Connection: Keep-Alive

{
    "requestURL": "/ISAPI/Event/triggers/notifications/channels/whiteLightAlarm",
    "statusCode": 4,
    "statusString": "Invalid Operation",
    "subStatusCode": "notSupport",
    "errorCode": 1073741825,
    "errorMsg": "notSupport"
}
```

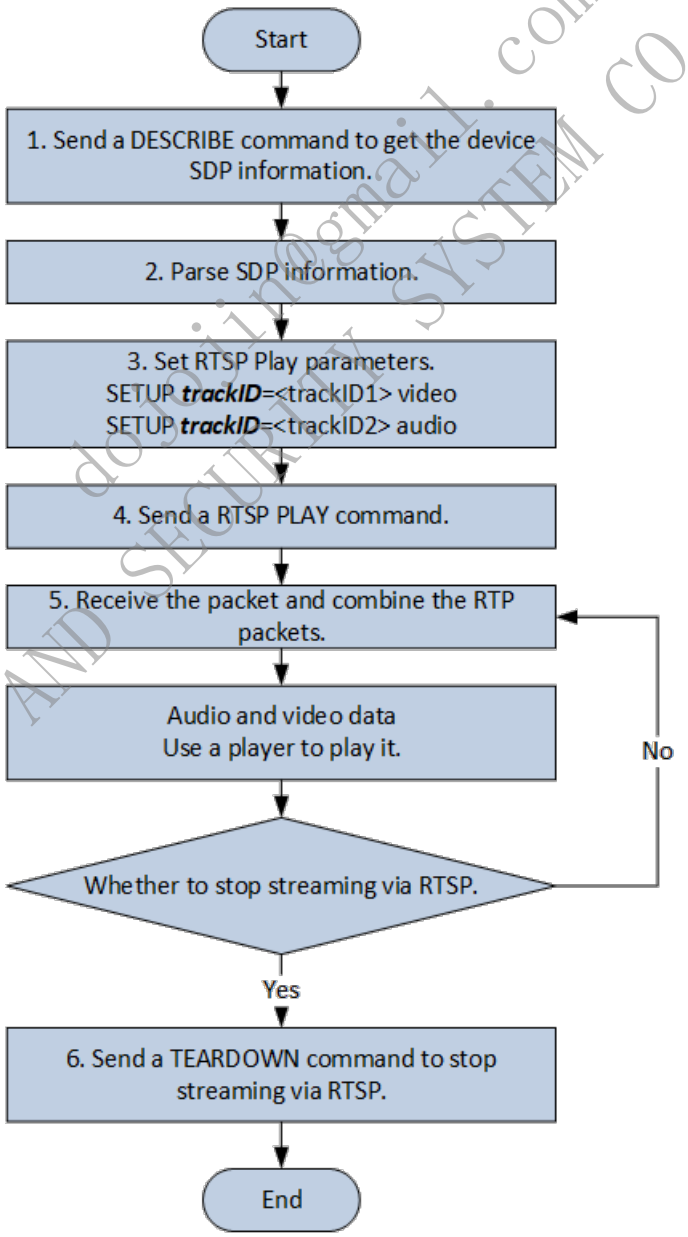
4.3 Real-Time Live View

4.3.1 Introduction to the Function

Supports getting and setting stream media parameters of devices such as resolution, coding format, and stream type.

Supports streaming from products via RTSP (Real Time Streaming Protocol, see details in RFC 7826).

4.3.2 API Calling Flow



VISION AND SECURITY SYSTEM CO.,LTD.
doc.qiu@gmail.com

1. A client sends RTSP DESCRIBE commands such as `DESCRIBE /ISAPI/Streaming/channels/101 RTSP/1.0`. Digest authentication with devices is required before this step. 2. The client parses the media SDP information returned by the device. 3. Set RTSP play parameters, that is to set the track ID parsed from SDP information via SETUP commands. For

example, trackID=1 indicates videos while trackID=2 indicates audios. 4. The client sends an RTSP PLAY command, and the device will send audio stream, video stream, and metadata in the format of `PLAY /ISAPI/Streaming/channels/101 RTSP/1.0`. 5. The client receives the RTP packet sent by the device. Divided RTP packets should be assembled on the client before being parsed. 6. The client sends the command RTSP TEARDOWN to stop streaming.

Notes:

- Digest authentication is required in RTSP playback. The method is the same as that of ISAPI digest authentication.
- The address format for streaming from devices is `rtsp:// <host>[:port]/ISAPI/Streaming/channels/<ID>`, of which `<host>` is the device IP address; `[:port]` is optional, and 554 by default; `<ID>` is the device channel ID * 100 + stream type (1-main stream, 2-sub-stream, 3-third stream). For example, the IP address of the target device is `172.7.203.11`, and the streaming address of main stream for channel 17 will be `rtsp://172.7.203.11:554/ISAPI/Streaming/channels/1701`.
- RTSP also supports containing user names and passwords in URL. The format is `rtsp://username:password@[address]:[port]/Streaming/Channels/[id](?parm1=value1&parm2-=value2…)`, such as `/Streaming/Channels/101?transportmode=unicast`.

4.3.3 Example

1. A client sends an RTSP DESCRIBE command.

```
DESCRIBE rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101 RTSP/1.0
CSeq:0
Accept:application/sdp
User-Agent:NKPlayer-1.00.00.081112
```

2. Server responds that authentication is required.

```
RTSP/1.0 401 Unauthorized
CSeq: 0
WWW-Authenticate: Digest realm="3521781c29acb312330dd668", nonce="026019333", algorithm="MD5"
```

3. The client sends an RTSP DESCRIBE request with authentication information again.

```
DESCRIBE rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101 RTSP/1.0
CSeq:1
Accept:application/sdp
Authorization: Digest username="admin", realm="3521781c29acb312330dd668", nonce="026019333", uri="rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101",
response="76a2c9c5b8edbd49838013cf1cf27941"
User-Agent:NKPlayer-1.00.00.081112
```

4. The device responds to SDP information.

VISION AND SECURITY SYSTEM CO.,LTD.
dojojin@gmail.com

```
RTSP/1.0 200 OK
CSeq: 1
Content-Type: application/sdp
Content-Length: 571
Date: Tue, 17 Nov 2020 02:09:45 GMT

v=0
o=- 1109162014219182 0 IN IP4 0.0.0.0
s= Media Server V4.22.126
i= Media Server Session Description : standard
e=NONE
c=IN IP4 0.0.0.0
t=0 0
a=control:*
b=AS:6154
a=range:npt=now-
m=video 0 RTP/AVP 96
i=Video Media
a=rtpmap:96 H264/90000
a=fmtp:96 profile-level-id=4D0014;packetization-mode=0
a=control:trackID=1
b=AS:6144
m=audio 0 RTP/AVP 8
i=Audio Media
a=rtpmap:8 PCMA/8000
a=control:trackID=2
b=AS:10
a=Media_header:MEDIAINFO=494D4B4802010000020000011171011040lF000000FA0000000000000000000000000000000000000000;
a=appversion:1.0
```

5. The client sends RTSP SETUP requests, and the server responds to them.

```
SETUP rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101/trackID=1 RTSP/1.0
CSeq:2
Authorization: Digest username="admin", realm="3521781c29acb312330dd668", nonce="026019333", uri="rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101",
response="ff343f5ff82deb028dd9b4932cc44201"
Transport:RTP/AVP/TCP;unicast;interleaved=0-1;ssrc=0
User-Agent:NKPlayer-1.00.00.081112
```

```
RTSP/1.0 200 OK
Session: 1127293610;timeout=60
Transport: RTP/AVP/TCP;unicast;interleaved=0-1;ssrc=433122aa
CSeq: 2
Accept-Ranges: NPT
Media-Properties: No-Seeking, Time-Progressing, Time-Duration=0
Date: Tue, 17 Nov 2020 02:09:45 GMT
```

```
SETUP rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101/trackID=2 RTSP/1.0
CSeq:3
Authorization: Digest username="admin", realm="3521781c29acb312330dd668", nonce="026019333", uri="rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101",
response="ff343f5ff82deb028dd9b4932cc44201"
Session:1127293610;timeout=60
Transport:RTP/AVP/TCP;unicast;interleaved=2-3;ssrc=0
User-Agent:NKPlayer-1.00.00.081112
```

```
RTSP/1.0 200 OK
Session: 1127293610;timeout=60
Transport: RTP/AVP/TCP;unicast;interleaved=2-3;ssrc=433122ab
CSeq: 3
Accept-Ranges: NPT
Media-Properties: No-Seeking, Time-Progressing, Time-Duration=0
Date: Tue, 17 Nov 2020 02:09:45 GMT
```

6. The client sends an RTSP PLAY request.

```
PLAY rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101 RTSP/1.0
CSeq:4
Authorization: Digest username="admin", realm="3521781c29acb312330dd668", nonce="026019333", uri="rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101",
response="24edf8a6ff3ef767f7c49d1c847200bd"
Session:1127293610;timeout=60
Range:npt=0.000000-0.000000
User-Agent:NKPlayer-1.00.00.081112
```

7. The server sends audio and video stream data.

dojojin@gmail.com VISION AND SECURITY SYSTEM CO.,LTD.

```
RTSP/1.0 200 OK
Session: 1127293610
CSeq: 4
Date: Tue, 17 Nov 2020 02:09:45 GMT

$. .....d1.w....c....".T....g....).i.........a....7.S..~J............X....X.
```

8. The client sends an RTSP TEARDOWN request, and the server responds to it.

```
TEARDOWN rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101 RTSP/1.0
CSeq:5
Authorization: Digest username="admin", realm="3521781c29acb312330dd668", nonce="026019333", uri="rtsp://10.21.84.147:554/ISAPI/Streaming/channels/101",
response="24edf8a6ff3ef767f7c49d1c847200bd"
Session:1127293610;timeout=60
Range:npt=0.000000-0.000000
User-Agent:NKPlayer-1.00.00.081112
```

```
RTSP/1.0 200 OK
Session: 1127293610
CSeq: 5
Date: Tue, 17 Nov 2020 02:09:50 GMT
```

5 Video (General)

5.1 Async Capturing Manually

5.1.1 Introduction to the Function

The client sends a capture command to the device, and the device immediately responds by returning the URL of the storage server where the captured picture is stored. The client can request image data from the storage server based on the received URL. It is suitable for the limited network bandwidth between the client and the device or the client centrally manages a large number of devices. The client can choose to use the image URL address carried in the alarm content to request pictures from the storage server when the network conditions improve or there is no concurrency of a large number of requests, so as to reduce the probability of image download failure and the risk of network congestion.

5.1.2 API Calling Flow

1. The client gets the system capability: `GET /ISAPI/System/capabilities`, and if `isSupportSnapshotAsync` is returned and its value is `true`, async manual capturing shall be supported.
2. (Optional) The client gets the async manual capturing capability of a specified channel: `GET /ISAPI/Streaming/channels/<channelID>/picture/async/capabilities?format=json`.
3. The client controls the specified channel to manually and asynchronously capture pictures: `GET /ISAPI/Streaming/channels/<channelID>/picture/async?format=json&imageType=<imageType>&URLType=<URLType>`, and the device returns the URL of the storage server for capturing pictures.

Remarks:

- If the step 2 returns a failure during the integration，it is recommended that you check whether the channel supports async manual capturing according to steps 1 and 3.

5.2 Manual Capturing Events

5.2.1 Introduction to the Function

According to the preset information and capture interval configured by the client, the device manually captures images at the specified preset and reports the image information in the form of events. It is used for the client to identify emergency situations and perform an emergency patrol on some presets for analysis. It has a higher priority than the daily scheduled patrol capture. Common usage scenarios: according to the scheduled capturing event (eventType: imageCapture) rules, the store camera regularly audits in a number of key areas. If in the event of an emergency, and a

doc.isp@gmail.com VISION AND SECURITY SYSTEM CO.,LTD.

worker wants to immediately re-inspect each area, you can set a manual capture rule to complete an independent audit. After the audit is completed, the camera will continue to audit according to the scheduled capturing rules.

5.2.2 API Calling Flow

1. The client gets the system capability: `GET /ISAPI/System/capabilities`, and if `isSupportManualImageCapture` returns `true`, manual capturing shall be supported.
2. The client gets manual capturing capability for a specified channel: `GET /ISAPI/System/Video/inputs/channels/<channelID>/manualImageCapture/capabilities?format=json`.
3. The client sets the parameters of manual capture events for a specified channel: Get parameters: `GET /ISAPI/System/Video/inputs/channels/<channelID>/manualImageCapture?format=json`. Set parameters: `PUT /ISAPI/System/Video/inputs/channels/<channelID>/manualImageCapture?format=json`.
4. Receive the event uploaded by the device in arming or listening mode: `eventType:manualImageCapture`.

Remarks:

- Each time the client sets manual capture event parameters, the channel immediately takes a patrol capture of the relevant presets. When the last preset capture is finished, the manual capture event ends.

5.3 Sync Capturing Manually

5.3.1 Introduction to the Function

The client sends a capture command to the device, and the device immediately returns the captured picture data in respond. It is often used in scenarios with high network bandwidth between clients and devices to facilitate the rapid transmission of image data, such as a LAN built on a company campus.

5.3.2 API Calling Flow

1. (Optional) The client gets the system capability: `GET /ISAPI/System/capabilities`, and if `isSupportManualSnapPicture` returns `true`, sync manual capturing shall be supported.
2. (Optional) The client gets sync manual capturing capability for a specified channel: `GET /ISAPI/Streaming/channels/<trackStreamID>/picture/capabilities?format=json`.
3. The client controls the specified channel to perform sync manual capturing: `GET /ISAPI/Streaming/channels/<trackStreamID>/picture?snapShotImageType=<snapShotImageType>&videoResolutionWidth=<videoResolutionWidth>&videoResolutionHeight=<videoResolutionHeight>&imageQuality=<imageQuality>&x=<x>&y=<y>&hight=<hight>&width=<width>`, and the device returns the captured binary picture data.

Remarks:

- If you only need to capture a JEPG picture, you can jump to step 3.
- If you need to capture pictures other than JEPG or capture in the designated area of the video, you need to perform steps 1 and 2 to determine whether the device supports it. If it does, go to step 3.
- In step 3, the value ranges of `videoResolutionWidth` and `videoResolutionHeight` are the same as `<videoResolutionWidth> and <videoResolutionHeight>` returned by `GET /ISAPI/Streaming/channels/<trackStreamID>/capabilities`.

6 VCA

6.1 Face Picture Library Management

6.1.1. Search via Picture Comparison in Face Capture Library

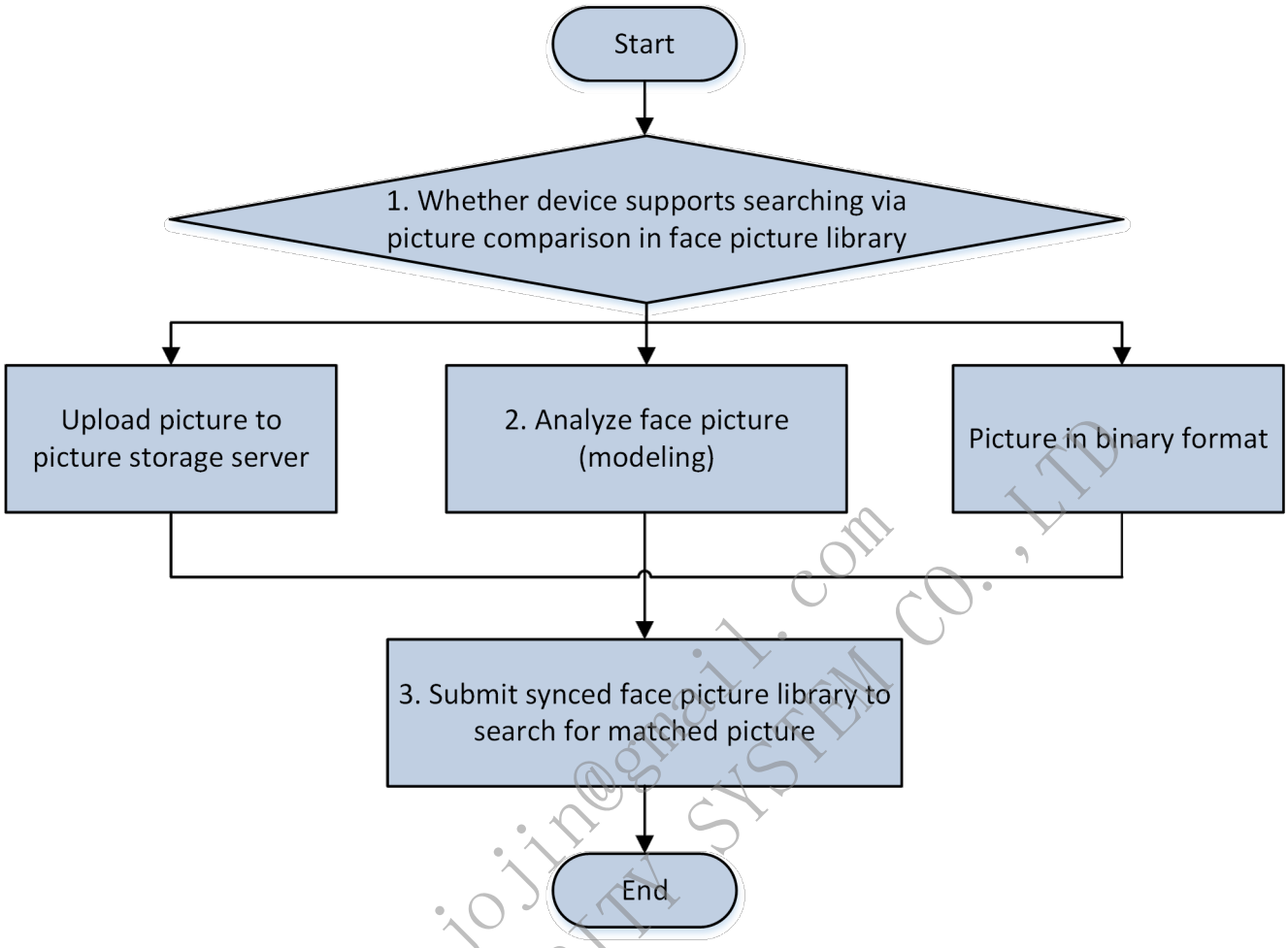
6.1.1.1 Introduction to the Function

VISION AND SECURITY SYSTEM CO.,LTD. dojojin@gmail.com

Picture comparison with captured picture library

Scene application: Search for similar person in face picture library, which can be used for identity confirmation.

6.1.1.2 API Calling Flow



dojojin@gmail.com VISION AND SECURITY SYSTEM CO.,LTD

1. **Check whether the device supports searching by picture synchronously.**

   Get the capability of searching via picture comparison: `GET /ISAPI/Intelligent/FDLib/capabilities?format=json`. If the node `isSuportFSSearchByPic` is returned and its value is `true`, it indicates that searching via picture comparison synchronously is supported.

   Note: For some devices supporting searching via picture comparison synchronously, the API for getting the capability is invalid, you can call `POST /ISAPI/Intelligent/FDLib/searchByPic?format=json` directly.

2. **Three methods are supported when searching picture.**

   By URL:

   Based on the picture storage server, picture can be uploaded and you can get the picture URL to download the picture file via the URL.

   By model data:

   Get face picture capability: `GET /ISAPI/SDT/Face/pictureAnalysis/capabilities`;

   Analyze (model) face pictures: `POSTT /ISAPI/SDT/Face/pictureAnalysis`; Return picture modeling data: `targetModelData`.

   By binary picture:

   Read binary picture files.

3. **Submit synced face picture library to use picture to search picture**

Search via picture comparison in face capture library: `POST /ISAPI/Intelligent/FDLib/searchByPic?format=json`;

Note: Input parameter node `dataType` include three modes. Picture URL corresponds to `faceURL`, model data corresponds to `targetModelData`, binary data mode is in form of form, and its JSON data is followed by binary picture data.

The request message example of binary picture data is as follows:

```
POST /ISAPI/Intelligent/FDLib/searchByPic?format=json
Host: device_ip:port
Accept-Language: zh-cn
Date: YourDate
Content-Type: multipart/form-data;boundary=<frontier>
Content-Length: text_length
Connection: keep-alive

--<frontier>
Content-Disposition: form-data; name=""
Content-Type: application/json

[JSON message]
--<frontier>
Content-Disposition: form-data; name="Picture_Name"
Content-Length: image_length
Content-Type: image/jpeg

[Picture data]
--<frontier>--
```

7 API Reference

7.1 Video (General)

7.1.1 Image Adjustment

7.1.1.1 Set the mounting scenario mode of a camera

Request URL

PUT /ISAPI/Image/channels/<channelID>/mountingScenario

Query Parameter

Parameter Name	Parameter Type	Description
channelID	string	--

Request Message

```
<?xml version="1.0" encoding="UTF-8"?>

<MountingScenario xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--req, object, attr:version{req, string, protocolVersion}-->
  <mode>
    <!--req, enum, "indoor,outdoor,day,night,morning,nightfall,mode1,mode2,mode3,mode4,highway,road2,faceSnap,backlight,frontlight","LowIllumination"-Low illumination,"backlight"-back light,"frontlight"-front light,"faceSnap"-face picture capture, subType:string, desc:“indoor“, “outdoor”, “day”, “night”, “morning”, “nightfall”, “mode1”, “mode2”, “mode3”, “mode4”, “street”, “LowIllumination”-Low illumination, “custom1”, “custom2”, “normal”, “road”, “faceSnap”-face picture capture, “highway”, “road2”, “backlight”-back light, “frontlight”-front light, “facenight”, “fastRoadAndLight”, “fastRoadNoLihgt”, “normalRoadAndLight”, “normalRoadNoLight”, “cityRoadAndBright”-->indoor
  </mode>
</MountingScenario>
```

Response Message

dojojin@gmail.com VISION AND SECURITY SYSTEM CO.,LTD.

```
<?xml version="1.0" encoding="UTF-8"?>

<ResponseStatus xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, response message, attr:version{ro, req, string, protocolVersion}-->
  <requestURL>
    <!--ro, req, string, request URL-->null
  </requestURL>
  <statusCode>
    <!--ro, req, enum, status code, subType:int, desc:0-OK, 1-OK, 2-Device Busy, 3-Device Error, 4-Invalid Operation, 5-Invalid XML Format, 6-Invalid XML
Content, 7-Reboot Required-->0
  </statusCode>
  <statusString>
    <!--ro, req, enum, status description, subType:string, desc:“OK” (succeeded), “Device Busy”, “Device Error”, “Invalid Operation”, “Invalid XML Format”,
“Invalid XML Content”, “Reboot” (reboot device)-->OK
  </statusString>
  <subStatusCode>
    <!--ro, req, string, sub status code, desc:sub status code-->OK
  </subStatusCode>
</ResponseStatus>
```

7.1.1.2 Get the capability of configurations of mounting scenario mode

Request URL

GET /ISAPI/Image/channels/<channelID>/mountingScenario/capabilities

Query Parameter

Parameter Name	Parameter Type	Description
channelID	string	--

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<MountingScenario xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, attr:version{req, string, protocolVersion}-->
  <mode
opt="indoor,outdoor,day,night,morning,nightfall,mode1,mode2,mode3,mode4,street,lowIllumination,custom1,custom2,normal,road,faceSnap,highway,road2,backlight,
frontlight,facenight,fastRoadAndLight,fastRoadNoLight,normalRoadAndLight,normalRoadNolight,riverSeaProtection,perimeterProtection,forestFirePrevention,foggy
,faceFirst,plateFirst,moveFirst,autoWDR,sportSnap,dustMoisture,rampWay">
    <!--ro, req, enum, Mode, subType:string, attr:opt{req, string}, desc:"lowIllumination"-Low illumination, "backlight"-back light, "frontlight"-front
light, "faceSnap"-face picture capture-->indoor
  </mode>
  <isSupportRecommendation>
    <!--ro, opt, bool-->true
  </isSupportRecommendation>
  <modeNameList size="2">
    <!--ro, opt, object, attr:size{req, int}-->
    <modeNameItem>
      <!--ro, opt, object-->
      <mode
opt="indoor,outdoor,day,night,morning,nightfall,mode1,mode2,mode3,mode4,street,lowIllumination,custom1,custom2,normal,road,faceSnap,highway,road2,backlight,
frontlight,facenight,fastRoadAndLight,fastRoadNoLight,normalRoadAndLight,normalRoadNolight,riverSeaProtection,perimeterProtection,forestFirePrevention,foggy
,faceFirst,plateFirst,moveFirst,autoWDR">
        <!--ro, req, enum, subType:string, attr:opt{req, string}-->indoor
      </mode>
      <modeName>
        <!--ro, req, string-->test
      </modeName>
    </modeNameItem>
  </modeNameList>
</MountingScenario>
```

7.1.1.3 Get the mounting scenario mode of a camera

Request URL

GET /ISAPI/Image/channels/<channelID>/mountingScenario?parameterType=<parameterType>

Query Parameter

VISION AND SECURITY SYSTEM CO.,LTD. zhoujin@gmail.com

Parameter Name	Parameter Type	Description
channelID	string	--
parameterType	string	Parameter type. When parameterType is recommendation, the obtained value is recommended by device. If parameterType is not configured, it will be the value of the parameter configured on the device.

Request Message

None

Response Message

```
<?xml version="1.0" encoding="UTF-8"?>

<MountingScenario xmlns="http://www.isapi.org/ver20/XMLSchema" version="2.0">
  <!--ro, req, object, attr:version{req, string, protocolVersion}-->
  <mode>
    <!--ro, req, enum, "indoor,outdoor,day,night,morning,nightfall,mode1,mode2,mode3,mode4,highway,road2,faceSnap,backlight,frontlight","lowIllumination"-
Low illumination,"backlight"-back light,"frontlight"-front light,"faceSnap"-face picture capture, subType:string, desc:“indoor”, “outdoor”, “day”, “night”,
“morning”, “nightfall”, “mode1”, “mode2”,“ mode3”, “mode4”, “highway”, “road2”, “faceSnap”, “backlight”, “frontlight”, “lowIllumination”-low illumination,
“backlight”-back light, “frontlight”-front light, “faceSnap”-face picture capture-->indoor
  </mode>
  <modeNameList>
    <!--ro, opt, array, subType:object-->
    <modeNameItem>
      <!--ro, opt, object-->
      <mode>
        <!--ro, req, enum, subType:string-->indoor
      </mode>
      <modeName>
        <!--ro, req, string-->test
      </modeName>
    </modeNameItem>
  </modeNameList>
</MountingScenario>
```

7.1.2 Manual Capture

7.1.2.1 Capture the picture in URL format

Request URL

POST /ISAPI/ContentMgmt/StreamingProxy/channels/<trackStreamID>/PictureByUrl?format=json

Query Parameter

Parameter Name	Parameter Type	Description
trackStreamID	string	--

Request Message

```
{
    "CaputreDescription": {
    /*opt, object*/
        "imageType":  "JPEG",
        /*req, enum, picture type, subType:string, desc:“JEPG”*/
        "URLType":  "cloudURL"
        /*req, enum, cloudURL, subType:string, desc:“cloudURL”-cloud storage or third party storage, “localURL”-device local storage*/
    }
}
```

Response Message

VISION AND SECURITY SYSTEM CO., LTD. dojojin@gmail.com

```
{
    "CaputreResult": {
    /*ro, opt, object*/
        "URLType":  "cloudURL",
        /*ro, req, string, cloudURL*/
        "imageURL":  "http://127.0.0.1"
        /*ro, req, picurl, picture URL*/
    }
}
```

7.2 VCA

7.2.1 Face Picture Library Management

7.2.1.1 Search face picture library information by arming mode

Request URL

POST /ISAPI/Intelligent/FDLib/search?format=json

Query Parameter

None

Request Message

```
{
    "searchResultPosition":  0,
    /*req, int, the start position of the search result in the result list, desc:in a single search, if you cannot get all the records in the result list, you can mark the end position and get the following records after the marked position in the next search*/
    "maxResults":  0,
    /*req, int, the maximum number of search results this time*/
    "libArmingType":  "armingLib"
    /*opt, enum, arming type, subType:string, desc:"armingLib" (armed library (default)), "nonArmingLib" (not armed library)*/
}
```

Response Message

```
{
    "responseStatusStrg":  "OK",
    /*ro, opt, enum, searching status description, subType:string, desc:"OK" (searching completed), "MORE" (searching for more data), "NO MATCH" (no matched data).*/
    "numOfMatches":  1,
    /*ro, opt, int, number of results returned this time*/
    "totalMatches":  1,
    /*ro, opt, int, total number of matched results*/
    "FDLib": [
    /*ro, opt, array, face picture library information, subType:object*/
        {
            "FDID":  "test",
            /*ro, opt, string, face picture library ID*/
            "faceLibType":  "blackFD",
            /*ro, opt, enum, face picture library type, desc:"blackFD" (list library), "staticFD" (static library)*/
            "name":  "test",
            /*ro, opt, string, name of the face picture library, range:[0,48]*/
            "customInfo":  "test",
            /*ro, opt, string, custom information, range:[0,192]*/
            "libArmingType":  "armingLib",
            /*ro, opt, enum, arming type, subType:string, desc:"armingLib" (armed library (default)), "nonArmingLib" (not armed library)*/
            "libAttribute":  "blackList"
            /*ro, opt, enum, library attribute type, subType:string, desc:"blackList" (blocklist), "VIP" (VIP library), "passerby" (passer-by library, which cannot be deleted)*/
        }
    ]
}
```

7.2.1.2 Delete all face picture libraries

Request URL

DELETE /ISAPI/Intelligent/FDLib?format=json

Query Parameter

None

Request Message

VISION AND SECURITY SYSTEM CO.,LTD. dojojin@gmail.com

None

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

7.2.1.3 Create a face picture library

Request URL

POST /ISAPI/Intelligent/FDLib?format=json

Query Parameter

None

Request Message

```
{
    "faceLibType":  "blackFD,staticFD",
    /*req, string, face picture library type, desc:face picture library type*/
    "name":  "test",
    /*req, string, name of the face picture library, range:[0,48], desc:name of the face picture library*/
    "customInfo":  "test",
    /*opt, string, custom information, range:[0,192], desc:custom information*/
    "libArmingType":  "armingLib",
    /*opt, enum, subType:string*/
    "libAttribute":  "blackList",
    /*opt, enum, subType:string*/
    "FDID":  "test"
    /*opt, string, face picture library ID, desc:face picture library ID*/
}
```

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, request URL*/
    "statusCode":  1,
    /*ro, req, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, req, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, req, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok",
    /*ro, opt, string, error details, desc:this node is required when the value of statusCode is not 1*/
    "FDID":  "test"
    /*ro, opt, string, face picture library ID, desc:face picture library ID*/
}
```

7.2.1.4 Delete a specified face picture library

Request URL

DELETE /ISAPI/Intelligent/FDLib?format=json&FDID=<FDID>&faceLibType=<faceLibType>&terminalNo=<terminalNo>

Query Parameter

VISION AND SECURITY SYSTEM CO.,LTD.
dojojin@gmail.com

Parameter Name	Parameter Type	Description
FDID	string	--
faceLibType	string	"blackFD" (list library), "staticFD" (static library)
terminalNo	string	Terminal No. It is an integer starting from 1.

Request Message

None

Response Message

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "OK",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok",
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
    "MErrCode":  "0x00000000",
    /*ro, opt, string*/
    "MErrDevSelfEx":  "0x00000000"
    /*ro, opt, string*/
}
```

7.2.1.5 Edit the information of a specific face picture library information, including name and custom information

Request URL

PUT /ISAPI/Intelligent/FDLib?format=json&FDID=<FDID>&faceLibType=<faceLibType>&terminalNo=<terminalNo>

Query Parameter

Parameter Name	Parameter Type	Description
FDID	string	--
faceLibType	string	--
terminalNo	string	--

Request Message

```
{
    "name":  "test",
    /*opt, string, face picture library name, range:[0,48]*/
    "customInfo":  "test",
    /*opt, string, custom information, range:[0,192]*/
    "libArmingType":  "armingLib",
    /*opt, enum, arming type of the list library, subType:string, desc:“armingLib” (armed face picture library), “nonArmingLib” (not armed face picture
library). The default value is "armingLib”*/
    "libAttribute":  "blackList"
    /*opt, enum, library type, subType:string, desc:“blackList” (blocklist library), “VIP” (VIP library), “passerby” (passerby library). The passerby
library cannot be deleted*/
}
```

Response Message

VISION AND SECURITY SYSTEM CO.,LTD. dojojin@gmail.com

```
{
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it corresponds to subStatusCode when statusCode is not 1*/
    "errorMsg":  "ok"
    /*ro, opt, string, error information, desc:this field is required when the value of statusCode is not 1*/
}
```

7.2.2 Face Picture Library Record Management

7.2.2.1 Get the total number of face records in all face picture libraries

Request URL

GET /ISAPI/Intelligent/FDLib/Count?format=json

Query Parameter

None

Request Message

None

Response Message

```
{
    "requestURL":  "http://10.7.52.31:8080/kms/services/rest/dataInfoService/downloadFile",
    /*ro, opt, string, request URL*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "ok",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, req, int, error code, desc:it is required when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok",
    /*ro, req, string, see the description of this node and above nodes in the message of JSON_ResponseStatus, desc:this node is required when the value of
statusCode is not 1*/
    "FDRecordDataInfo": [
    /*ro, opt, array, information of face records in face picture library, subType:object, desc:this node is valid when errorCode is 1 and errorMsg is
"ok"*/
        {
            "FDID":  "test",
            /*ro, opt, string, face picture library ID, desc:the maximum size is 63 bytes*/
            "faceLibType":  "blackFD",
            /*ro, opt, enum, face picture library type, subType:string, desc:face picture library type "blackFD" list library,"staticFD" static library, the
maximum size is 32 bytes*/
            "name":  "test",
            /*ro, opt, string, face picture library name, desc:the maximum size is 48 bytes*/
            "recordDataNumber":  123,
            /*ro, opt, int, number of records*/
            "libArmingType":  "armingLib",
            /*ro, opt, enum, arming type, subType:string, desc:arming type*/
            "libAttribute":  "general",
            /*ro, opt, enum, subType:string*/
            "personnelFileEnabled":  true
            /*ro, opt, bool*/
        }
    ],
    "totalRecordDataNumber":  500000,
    /*ro, opt, int, total number of records, desc:total number of records*/
    "FDCapacity": {
    /*ro, opt, object*/
        "total":  0.000,
        /*ro, req, float*/
        "use":  0.000,
        /*ro, req, float*/
        "remain":  0.000,
        /*ro, req, float, remaining space, desc:remaining space*/
        "maxRecordDataNumber":  0,
        /*ro, opt, int*/
        "useRecordDataNumber":  0,
        /*ro, opt, int*/
        "remainRecordDataNumber":  0
        /*ro, opt, int*/
    }
}
```

VISION AND SECURITY SYSTEM CO.,LTD.
dojo.jin@gmail.com

7.2.2.2 Get the total number of face records in a face picture library

Request URL

GET /ISAPI/Intelligent/FDLib/Count?format=json&FDID=<FDID>&faceLibType=<faceLibType>&terminalNo=<terminalNo>

Query Parameter

Parameter Name	Parameter Type	Description
FDID	string	--
faceLibType	string	"blackFD" (list library), "staticFD" (static library), "infraredFD" (IR face picture library)
terminalNo	string	--

Request Message

None

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, request URL*/
    "statusCode":  1,
    /*ro, opt, int, status code, desc:1 (succeeded). It is required when an error occurred*/
    "statusString":  "OK",
    /*ro, opt, string, status description, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "subStatusCode":  "ok",
    /*ro, opt, string, sub status code, range:[1,64], desc:"ok" (succeeded). It is required when an error occurred*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:it is required when the value of statusCode is not 1, it corresponds to subStatusCode*/
    "errorMsg":  "ok",
    /*ro, opt, string, error description, desc:this node is required when the value of statusCode is not 1*/
    "FDID":  "test",
    /*ro, opt, string, face picture library ID, range:[1,64]*/
    "faceLibType":  "blackFD",
    /*ro, opt, enum, face picture library type, subType:string, desc:face picture library type*/
    "name":  "test",
    /*ro, opt, string, name of the face picture library, desc:the maximum length is 48*/
    "recordDataNumber":  123
    /*ro, opt, int, total number of records*/
}
```

7.2.2.3 Add a face record to the face picture library

Request URL

POST /ISAPI/Intelligent/FDLib/FaceDataRecord?format=json

Query Parameter

None

Request Message

VISION AND SECURITY SYSTEM CO.,LTD.
doJojin@gmail.com

```
{
    "faceURL":  "test",
    /*opt, string, picture storage URL inputted when uploading the face picture by URL, desc:select this field or modelData*/
    "faceLibType":  "blackFD",
    /*req, enum, face picture library type, subType:string, desc:face picture library type*/
    "FDID":  "test",
    /*req, string, face picture library ID, range:[1,64]*/
    "FPID":  " test",
    /*opt, string, face record ID, range:[1,1000], desc:face record ID,it can be generated by device or inputted. If it is inputted,it should be the unique ID with the combination of letters and digits,and the maximum length is 63 bytes; if it is generated by the device automatically,it is the same as the employee No. (person ID)*/
    "name":  "张三",
    /*req, string, name of person in the face picture, range:[1,1000], desc:the maximum size is 96 bytes*/
    "gender":  "male",
    /*opt, enum, gender of person in the face picture, subType:string, desc:“male”, “female”, “unknown”; the maximum size is 32 bytes*/
    "bornTime":  "2004-05-03",
    /*req, string, birthday of person in the face picture, desc:the maximum size is 20 bytes*/
    "city":  "130100",
    /*opt, string, city code of birth for the person in the face picture, desc:the maximum size is 32 bytes*/
    "certificateType":  "ID",
    /*opt, enum, certificate type, subType:string, desc:"ID" (ID card), "officerID" (officer ID), "other" (other type), "LawyerCertificate" (lawyer qualification), "birthCertificate", "householdRegister", "judgeCertificate", "studentIDCard", "workCard" (work permit),  "policeID", "foreignPermanentResidentIDCard", "passport", "pressCard", "prosecutorCard", "temporaryIDCard", "temporaryResidencePermit"*/
    "certificateNumber":  "test",
    /*opt, string, certificate No., range:[1,64], desc:the maximum size is 32 bytes*/
    "caseInfo":  "test",
    /*opt, string, case information, desc:the maximum size is 192 bytes; it is valid when faceLibType is "blackFD”*/
    "tag":  "aa,bb,cc,dd",
    /*opt, string, custom tag, desc:up to 4 tags, which are separated by commas. The maximum size is 195 bytes. It is valid when faceLibType is "blackFD”*/
    "address":  "test",
    /*opt, string, person address, desc:the maximum size is 192 bytes. It is valid when faceLibType is "staticFD”*/
    "customInfo":  "test",
    /*opt, string, custom information, desc:the maximum size is 192 bytes. It is valid when faceLibType is "staticFD"*/
    "modelData":  "test",
    /*opt, string, target modeling data, desc:target model data,non-modeled binary data needs to be encrypted by Base64 during transmission*/
    "transfer":  true,
    /*opt, bool, whether to enable transfer, desc:whether to enable transfer*/
    "operateType":  "byTerminal",
    /*opt, enum, operation type, subType:string, desc:"byTerminal” (operate by terminal)*/
    "terminalNoList": [1],
    /*opt, array, terminal ID list, subType:int, desc:this node is required when operation type is "byTerminal"; currently, only one terminal is supported*/
    "PicFeaturePoints": [
    /*opt, array, feature points to be applied, subType:object*/
        {
            "featurePointType":  "face",
            /*req, enum, feature point type, subType:string, desc:"face", "leftEye" (left eye), "rightEye" (right eye), "leftMouthCorner" (left corner of mouth), "rightMouthCorner" (right corner of mouth), "nose"*/
            "coordinatePoint": {
            /*opt, object, coordinates of the feature point, desc:object,coordinates of the feature point*/
                "x":  1,
                /*req, int, X-coordinate, range:[0,1000], desc:normalized X-coordinate which is between 0 and 1000*/
                "y":  1,
                /*req, int, Y-coordinate, range:[0,1000], desc:normalized Y-coordinate which is between 0 and 1000*/
                "width":  1,
                /*opt, int, width, range:[0,1000], desc:this node is required when featurePointType is "face"*/
                "height":  1
                /*opt, int, height, range:[0,1000], desc:this node is required when featurePointType is "face"*/
            }
        }
    ],
    "cutOutRect": {
    /*opt, object, picture cropping frame, desc:the origin is the upper-left corner of the screen, which is used for the target frame in matting*/
        "x":  0.000,
        /*req, float, x-coordinate, range:[0,1], desc:normalize coordinate*/
        "y":  0.000,
        /*req, float, y-coordinate, range:[0,1], desc:normalize coordinate*/
        "width":  0.000,
        /*req, float, width, range:[0,1]*/
        "height":  0.000
        /*req, float, height, range:[0,1]*/
    },
    "faceType":  "normalFace",
    /*opt, enum, face picture type, subType:string, desc:"normalFace" (normal face, default value), "patrolFace" (patrol person face), "hijackFace" (duress person face), "superFace" (super person face)*/
    "saveFacePic":  true,
    /*opt, bool, whether to save face pictures*/
    "leaderPermission": [1, 2, 3, 4]
    /*opt, array, first authentication permission, subType:int, range:[1,4], desc:if the array is empty, it indicates clearing first authentication permission of the person; if not, it indicates that the person have first authentication permission for the door 1, 2, 3, and 4*/
}
```

Parameter Name	Parameter Value	Parameter Type(Content-Type)	Content-ID	File Name	Description
faceURL	[Message content]	application/json	--	--	--
img	[Binary picture data]	image/jpeg		facePic.jpg	--

VISION AND SECURITY SYSTEM CO., LTD. dojojin@gmail.com

Note: The protocol is transmitted in form format. See Chapter 4.5.1.4 for form framework description, as shown in the instance below.

```
--<frontier>
Content-Disposition: form-data; name=Parameter Name;filename=File Name
Content-Type: Parameter Type
Content-Length: ****
Content-ID: Content ID
Parameter Value
```

- Parameter Name: the name property of Content-Disposition in the header of form unit; it refers to the form unit name.
- Parameter Type (Content-Type): the Content-Type property in the header of form unit.
- File Name (filename): the filename property of Content-Disposition of form unit Headers. It exists only when the transmitted data of form unit is file, and it refers to the file name of form unit body.
- Parameter Value: the body content of form unit.

Response Message

```
{
    "requestURL":  "test",
    /*ro, opt, string, request URL*/
    "statusCode":  1,
    /*ro, req, int, status code*/
    "statusString":  "test",
    /*ro, req, string, status description*/
    "subStatusCode":  "test",
    /*ro, req, string, sub status code*/
    "errorCode":  1,
    /*ro, opt, int, error code, desc:this field is required when the value of statusCode is not 1*/
    "errorMsg":  "ok",
    /*ro, opt, string, error description, desc:see the description of this node and above nodes in the message of JSON_ResponseStatus*/
    "FPID":  "test",
    /*ro, opt, string, face record ID, range:[1,64], desc:face record ID returned when the face record is added,it is unique,and the maximum size is 63 bytes. This node is valid when errorCode is "1" and errorMsg is "ok"*/
    "rowKey":  "test"
    /*ro, opt, string, rowKey of face record, desc:rowKey of face record is returned after face data is added. It is unique and in string format. The maximum length is 64 bytes.*/
}
```

8 How-To Video Guidance

If you need access to corresponding video guidance for device integration, please register on https://tpp.hikvision.com and visit our Training Center: https://tpp.hikvision.com/tpp/Training. The Training Center is specifically designed to provide technical training and guidance resources for our partners. On this platform, you can find integration video tutorials for various devices, enabling better understanding and learning of the integration process. To offer more personalized service, our Training Center also supports filtering by integration protocols, devices, and applications.

VISION AND SECURITY SYSTEM CO.,LTD. dojojin@gmail.com